

Guía de Preguntas de Entrevista — Semana 1

Bootcamp: De Cero a Data Engineer - Databricks SQL

Bloque temático: Fundamentos Databricks SQL | Clase Teórica + Clase Grupal

Instructor: Luciano Argolo

SEMANA 1

DATABRICKS SQL

ENTREVISTAS TÉCNICAS

Estas preguntas cubren los temas técnicos de la Semana 1 y situaciones reales discutidas en la clase grupal. En una entrevista técnica o conductual, te van a complicar progresivamente. La clave no es saber la respuesta perfecta, sino mostrar **cómo pensás**.

Rol y Plataforma

Data Engineer, Data Analyst,
Data Scientist y Databricks

Metodología

Pensar en Datos y pensamiento
estructurado

Manejo de Crisis

Sin contexto, sin equipo, sin
analista

SQL Base

SELECT, WHERE, GROUP BY, HAVING, agregaciones y
JOINS

Negocio

Métricas, contexto y capa transaccional vs
analítica

Rol del Data Engineer y Plataforma

Pregunta 1: ¿Cuál es la diferencia entre Data Engineer, Data Analyst y Data Scientist? ¿Por qué nació el DE como rol separado?

Data Engineer (DE) Ingesta, transformación, modelado, orquestación y calidad del dato. Es el rol central que alimenta a todos los demás.	Data Analyst (DA) Analiza datos ya preparados, crea dashboards y reportes. Pregunta: <i>¿qué pasó?</i>	Data Scientist (DS) Modelos predictivos y machine learning. Pregunta: <i>¿qué va a pasar?</i>



SQL y Python son transversales a los tres roles. El DE nació porque el Data Scientist dedicaba el **70-80% de su tiempo** a preparar datos en vez de modelar. Los primeros DEs son científicos de datos reconvertidos.

Clave para entrevistas: Pregunta de apertura clásica. Si podés explicar dónde empieza y termina tu responsabilidad respecto a los otros roles, demostrás claridad profesional.

Pregunta 2: ¿Qué es Databricks y por qué un Data Engineer lo usaría?

Databricks es una **plataforma unificada de datos** que permite escribir y ejecutar SQL, Python y otros lenguajes desde un mismo entorno. Elimina semanas de configuración de infraestructura (Docker, Airflow, PostgreSQL, dependencias). Con la **Free Edition** se puede empezar a correr queries en minutos, sin tarjeta de crédito.



Workspace

Carpetas y organización del entorno de trabajo



Notebooks

Donde se escribe y ejecuta código. Soporta `%sql`, `%python`, `%sh`



Cluster

Motor de cómputo que procesa las queries y transformaciones



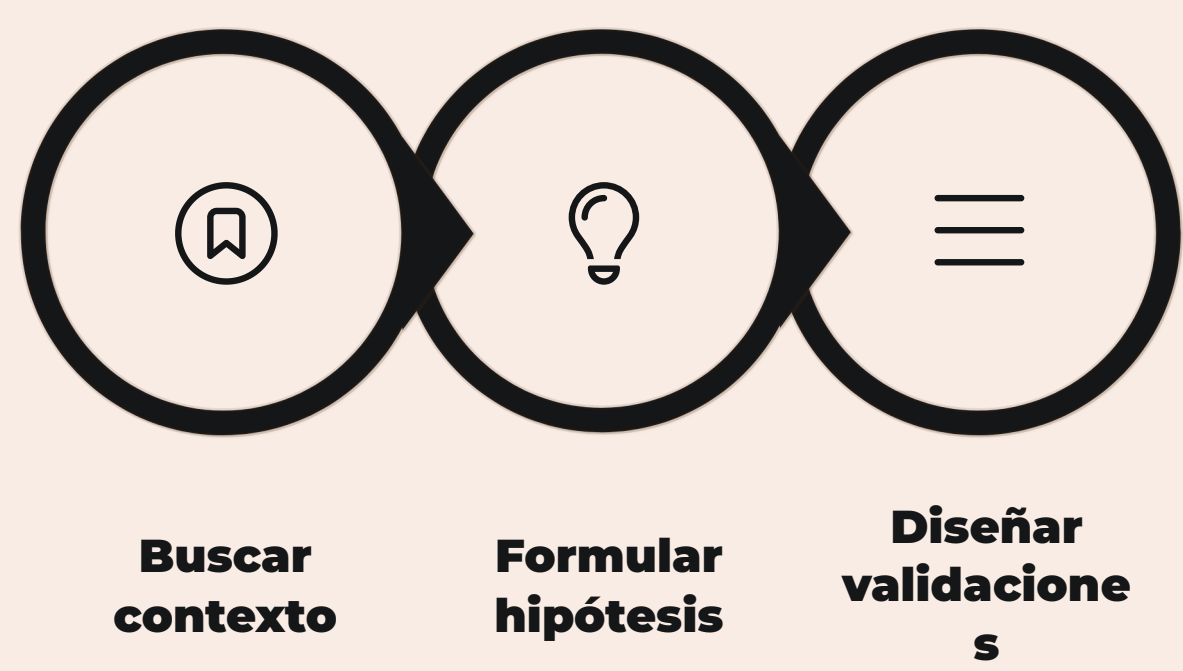
Catalog

Explorar tablas, ver tipos de datos y hacer linaje de datos

Clave para entrevistas: Si te preguntan "¿con qué herramientas trabajás?", poder describir qué hace cada componente de Databricks muestra que no solo sabés correr queries – entendés el entorno donde vivís como ingeniero.

Metodología y Pensamiento Estructurado

Pregunta 3: ¿En qué momento se formula la hipótesis?



Si arrancás a codear sin hipótesis, hacés exploraciones sin rumbo. Y si formulás la hipótesis **después** de ver los resultados, la vas a acomodar a lo que encontraste – ahí empiezan los sesgos.

Clave para entrevistas: Demuestra que pensás antes de codear. *"Primero la pregunta, después la query"* es la frase que resume todo el método de 5 pasos.

Pregunta 4: Te dan acceso a una tabla nueva que no conocés. ¿Cuál es tu paso a paso para entenderla antes de escribir queries de negocio?

01	02	03
DESCRIBE	SELECT * LIMIT 10	COUNT(*)
Ver columnas, tipos de datos y estructura de la tabla	Ver datos reales y entender qué contiene cada campo	Saber el volumen total de la tabla
04	05	06
Revisar nulos	Explorar distribución	Verificar rangos
COUNT de campos clave vs COUNT(*) para detectar datos faltantes	GROUP BY en campos clave para entender cardinalidad	MIN, MAX, AVG en campos numéricos para detectar outliers
07		
Formular hipótesis		
Antes de hacer queries de negocio, definir qué esperás encontrar		

Clave para entrevistas: Demuestra pensamiento estructurado usando herramientas concretas. Cada paso usa algo que aprendiste en esta semana (DESCRIBE, SELECT, COUNT, GROUP BY, MIN/MAX/AVG).

Manejo de Crisis: Falta de Contexto

Pregunta 5: No tenés analista funcional, no tenés analista de negocio, y echaron a tu jefe. ¿Cómo conseguís contexto?

1

Buscar documentación existente

Revisar si dejaron documentación técnica, wikis, confluence, o cualquier registro del sistema anterior.

2

Ir a los dueños del dato

La persona del negocio que **necesita** esos datos. Ellos saben qué esperan ver aunque no sepan cómo se construye.

3

Hablar con stakeholders

Jefe del lado del cliente, personas operativas que cargan los datos. Siempre hay alguien que sabe algo parcial.

4

Convertirte en el analista funcional

Si nadie sabe, **vos investigás, preguntás y reconstruís**. No quedarse paralizado.

⚠ **Clave:** No quedarse paralizado. Siempre hay alguien que sabe algo parcial. Esto no lo enseña ningún curso – se aprende en situaciones reales. Si en la entrevista te sacan al analista, al jefe, al acceso, no te frustres. Eso es intencional. Quieren ver que seguís pensando.

SQL: Agregaciones, GROUP BY y HAVING

Pregunta 6: Tenés esta query y falla. ¿Qué está mal y cómo la corregís?

❌ Query con errores

```
SELECT pickup_zip, AVG(trip_distance) AS avg_dist
FROM samples.nyctaxi.trips
GROUP BY pickup_zip
WHERE avg_dist > 5
```

Errores encontrados

- **Error 1:** WHERE va **antes** de GROUP BY en el orden de la query, no después.
- **Error 2:** WHERE **no puede** filtrar sobre campos agregados. Para filtrar grupos después de agregar, se usa HAVING.

✅ Query corregida

```
SELECT pickup_zip, AVG(trip_distance) AS avg_dist
FROM samples.nyctaxi.trips
GROUP BY pickup_zip
HAVING avg_dist > 5
```

📄 Orden de ejecución de SQL: FROM → WHERE → GROUP BY → funciones de agregación → HAVING → SELECT → ORDER BY

Pregunta 7: ¿Cuál es la diferencia entre WHERE y HAVING?

WHERE Filtra filas individuales antes de agrupar. Opera sobre datos crudos, fila por fila. No puede usar funciones de agregación como COUNT(), AVG(), MAX(). Un alias creado con AS en el SELECT no se puede usar en el WHERE.	HAVING Filtra grupos después de agregar. Opera sobre los resultados del GROUP BY. Puede usar funciones de agregación y alias definidos en el SELECT.
--	---

Clave para entrevistas: La respuesta corta: *"WHERE filtra filas antes de agrupar, HAVING filtra grupos después de agregar"*. Si además mencionás el orden de ejecución, demostrás que no memorizaste – entendés por qué.

Pregunta 8: ¿Qué significa que al hacer GROUP BY "se pierde granularidad"? ¿Qué implicancias tiene?

Al agrupar por zona y sacar MAX del precio, **perdemos los registros individuales**. Ya no podemos saber cuál fue el viaje específico que tuvo ese precio máximo. Solo quedan los valores agregados por grupo.

📄 **Regla de oro del GROUP BY:** Cualquier columna en el SELECT que **no** esté dentro de una función de agregación **debe** aparecer en el GROUP BY. Si necesitás datos individuales Y datos agregados al mismo tiempo, GROUP BY solo no alcanza – en Semana 2 se ven Window Functions que resuelven eso.

Clave para entrevistas: Si te preguntan "¿qué perdés al agrupar?", la respuesta es granularidad. Mostrar que entendés las limitaciones de GROUP BY y que sabés que hay herramientas más avanzadas es un diferenciador.

JOINS: Conectando Tablas y Diagnóstico

Pregunta 9: ¿Cómo encontrarías clientes que nunca hicieron una compra?

Solución con LEFT JOIN + IS NULL

```
SELECT c.nombre, c.email
FROM clientes c
LEFT JOIN ordenes o
  ON c.cliente_id = o.cliente_id
WHERE o.orden_id IS NULL
```

¿Por qué funciona?

- El LEFT JOIN trae todos los clientes, tengan o no órdenes.
- Los que no tienen órdenes aparecen con NULLs en las columnas de la tabla derecha.
- Filtrando por IS NULL, te quedás solo con los que no tienen match.

Clave para entrevistas: Patrón clásico de entrevista SQL. "LEFT JOIN + IS NULL" es la forma estándar de encontrar registros sin relación. Si lo explicás sin dudar, demostrás que manejas JOINS.

Pregunta 10: Volvés de vacaciones y los datos están triplicados. Nadie sabe qué pasó. ¿Cómo resolvés?

1 Revisar los JOINS

Un JOIN sobre un campo no único multiplica registros. Si una tabla tiene duplicados en la columna de JOIN, cada fila de la otra tabla se cruza con cada duplicado.

2 Buscar desde cuándo empezó

Comparar datos por fecha para acotar el rango del problema y encontrar el punto de quiebre.

3 Revisar la primary key

Si no hay PK definida, es más fácil que entren duplicados sin que nadie lo detecte.

4 Verificar las tablas de origen

¿Los datos ya venían duplicados o se generó en la transformación? Rastrear el origen exacto del problema.

✓ **Caso real:** Un JOIN se hacía sobre un campo que empezó a triplicarse en una tabla de otro equipo. Se resolvió haciendo JOIN con más campos únicos.

Clave para entrevistas: 9 de cada 10 veces que los datos se multiplican, el problema está en un JOIN mal diseñado. Arrancá siempre por ahí.

Métricas, Contexto de Negocio y Arquitectura

Pregunta 11: Te piden construir un dashboard pero nadie te definió qué métricas necesitan. ¿Qué hacés?

→ No arrancar a construir sin saber qué se necesita

Un dashboard sin métricas definidas es un gráfico lindo que nadie va a usar.

→ Preguntar al negocio

¿Cuál es el dato principal que te dice si van bien o mal? Ejemplo: un local de lámparas – la métrica clave no es la cantidad de empleados (indicador indirecto), sino las **ventas netas vs costos**.

→ Entender el impacto del DE en las métricas

Construimos las tablas y métricas que alimentan los dashboards. Si no entendés qué mide el negocio, vas a limpiar datos que nadie usa y dejar sucios los que importan.

Clave para entrevistas: Mostrar que antes de codear preguntás "¿qué necesita el negocio?" separa a un técnico de un profesional. El DE que entiende las métricas del negocio construye cosas que la empresa realmente usa.

Pregunta 12: ¿Por qué se separa la capa transaccional de la capa analítica?

Transaccional (OLTP)

Donde ocurren las operaciones en tiempo real: compras, pagos, transferencias. Tiene que ser **rápida** y disponible 24/7.



Analítica (OLAP)

Donde se responden preguntas estratégicas: "*¿qué región vendió más en 6 meses?*". Mismos datos, **modelos distintos**.

⚠ Si 400 personas corren queries analíticas sobre la capa transaccional, **la ralentizan y afectan las operaciones** del negocio. Por eso se separan: mismos datos, modelos distintos, para no interferir con las operaciones.

Clave para entrevistas: Concepto fundamental de arquitectura de datos. En semanas siguientes vas a ver cómo esta separación se materializa en capas concretas (Bronze, Silver, Gold), pero el principio de "*no mezclar operación con análisis*" es lo primero que tenés que poder explicar.

Puntos Clave: Qué Evalúan en una Entrevista

Estas preguntas no buscan la respuesta perfecta. Buscan ver **cómo pensás bajo presión**.

Lo que evalúan	Cómo demostrarlo
Pensamiento estructurado	Arrancar por contexto e hipótesis, ANTES de codear
Claridad sobre tu rol	Saber dónde empieza y termina tu responsabilidad vs DA/DS
Manejo de crisis	No paralizarte. Buscar opciones, escalar, preguntar
SQL sólido	WHERE vs HAVING, JOINS, granularidad, IS NULL
Contexto de negocio	Preguntar por métricas antes de construir. Entender transaccional vs analítico
Comunicación	Explicar tu razonamiento paso a paso, no saltar al código

- ✅ **Consejo final:** Si en la entrevista te complican progresivamente (te sacan al analista, al jefe, al acceso), no te frustres. Eso es intencional. Quieren ver que seguís pensando. Cada complicación es una oportunidad de mostrar otro ángulo de tu pensamiento.